



Welcome to

0. Introduction

Secure Development Practices

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, <https://codeberg.org/kramse/ek-SDL-practices.tex> in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Contact information



- Henrik Kramselund, he/him internet samurai mostly networks and infosec
- Network and security consultant Zencurity, teacher and activist
- Master from the Computer Science Department at the University of Copenhagen (DIKU)
- Email: hk@zencurity.com Mobile: +45 2026 6000
- Run a small network for fun AS57860

You are welcome to drop me an email

Goals for today

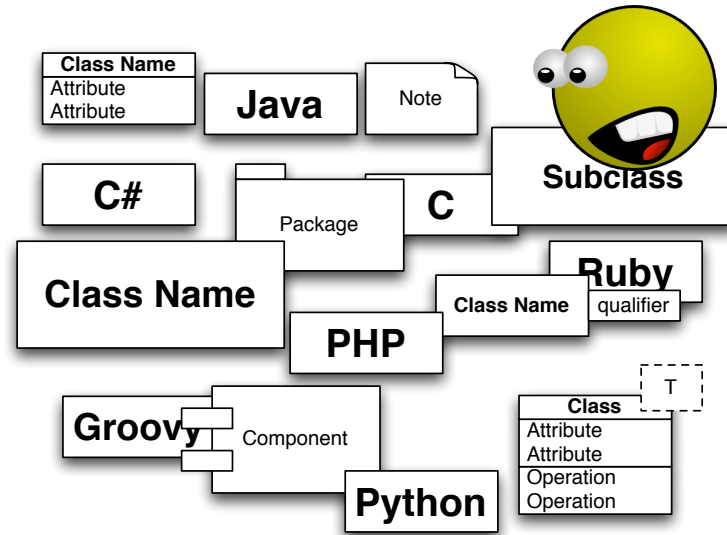


Today's goals:

- Security patterns, insecure and secure design
- What is SDL, SDLC, SSDL – multiple names, but essentially the same
- Security Testing Versus Traditional Software Testing
- Standards ISO27001/CIS Controls, Application Software Security

Photo by Thomas Galler on Unsplash

Software Security



Software security - because software is everywhere

Give a high level view – connect things

Contents



- Software security in general – how do things fail
- History, cases, vulnerabilities, falsehoods and assumptions
- Security principles: least privilege, fail-safe defaults, etc.
- OWASP Top 10, CVE/CWE and "Monster mitigations"
- Security patterns, insecure and secure design
- Policies, coding standards, compiler warnings, git hooks, defensive programming
- Code-Auditing Strategies: static analysis and dynamic analysis
- Secure Development Lifecycle (SDL, SDLC, SSDL) – multiple names, but essentially the same
- Optional: Exercises, questions

Day 1 will be Advanced subjects and details

Time schedule



Plan for the day

12:45 - 13:30 – Session 1: How does *bad software* happen?!

13:45 - 14:30 – Session 2: Security principles and design

14:45 - 15:30 – Session 3: Code auditing, analysis and security testing

15:30 - 16:30 – Session 4: Coding standards, Git hooks and scanner tools

16:30 - 16:45 – Exercises, questions

Session 0: How software fails



- Software security in general – how do things fail
- History, cases, vulnerabilities, falsehoods and assumptions
- What is a CVE
- Exercise: browse CVE sites, find vulnerabilities in software you use

Computer worms



A *computer worm* is a program that copies itself from one computer to another.

Source: *Computer Security: Art and Science*, Matt Bishop, ISBN 9780321712332

- Computer worms have existed since research began in the mid-1970s
- Morris Worm from November 2, 1988 was a famous example
- Often quoted: 6,000 of 60,000 computers infected – but this was a guess, other estimates are 1,000–3,000
- Led to the creation of CERT/CC, and Morris was the first person tried under the Computer Fraud and Abuse Act
- Virus, trojan or worm?
Unless you work specifically in the computer virus industry, call it all malware

The Internet Worm 2 November 1988



Vulnerability	How the worm abused it
Buffer overflow in fingerd	VAX code sent to overflow the buffer
Sendmail	DEBUG functionality
Trust between systems: rsh, rexec, ...	Found systems to infect in /etc/hosts.equiv, .rhosts, .forward, netstat ...
Bad passwords	Password cracking using internal list of 432 words and /usr/dict/words

Contained camouflage! Program name set to 'sh', used fork() to switch PID regularly

Made by Robert T. Morris, Jr.

Source: *The Internet Worm Program: An Analysis*, Eugene H. Spafford,
Purdue Technical Report CSD-TR-823, 1988

<https://spaf.cerias.purdue.edu/tech-reps/823.pdf>

Stuxnet



Worm in 2010 intended to infect Iran nuclear program

Target was the uranium enrichment process

Infected other industrial sites

SCADA, and Industrial Control Systems (ICS) are becoming very important for whole countries

A small *community* of consultants works in these *isolated* networks, but can be used as infection vector - they visit multiple sites

More information can be found in <https://en.wikipedia.org/wiki/Stuxnet>

Mentioned here as an example of airgapped/isolated control networks

Current events



Current events are always relevant, as they happen – right NOW!

- Fortinet CVEs - summary, make this an exercise, evaluate a new product suggested by your boss
- VPN – protocols belong in network courses, but implementation is at least as important?
- NIS2 – impact on "software security", can you run an open source project with high risk CVEs?
- Citrix ADC load balancers are popular in Denmark
- OpenSSH even has security vulnerabilities – coming from the OpenBSD project which focuses on security
- CPE routers hacked and becoming botnet devices

This slide was written a long time ago! This last month both NetScaler and OpenBSD had new vulns!

Vulnerabilities - CVE



Common Vulnerabilities and Exposures (CVE):

- classification
- identification

When discovered each vuln gets a CVE ID

CVE maintained by MITRE - not-for-profit org for research and development in the USA.

National Vulnerability Database search for CVE.

Sources: <https://www.cve.org/> and <https://nvd.nist.gov>

Also check out OWASP Top 10 <https://owasp.org/>

Current CVEs



We will be talking about software "problems" often named vulnerabilities, with CVE IDs Here are some example vulnerabilities Rezilion identified back in 2023:

- CVE-2026-45586 – CTFMON EoP, public zero-day
- CVE-2026-41089 – Netlogon RCE CVSS 9.8, disclosed May, confirmed exploited in June
- CVE-2026-56155 – AD FS EoP, exploited zero-day
- CVE-2026-56164 – SharePoint EoP, exploited zero-day
- CVE-2026-81963 – Windows Update Stack EoP, CVSS 7.8, on CISA KEV
- CVE-2026-85880 – Windows ALPC heap overflow EoP, CVSS 7.8, AppContainer sandbox escape, on CISA KEV
- CVE-2026-69730 – Windows DNS Server RCE, use-after-free, CVSS 9.8

Note: we will NOT dive deep into these, just acknowledge that they exist

Microsoft Patch Tuesday: June–September 2026



Month	CVEs (Microsoft)	Critical	Zero-days	Exploited
June	198–208	32–37	3	0 at release*
July	569–570	56–59	3	2
August	398–421	42–62	1–3	1
September	964–974	50–113	2	2

- Vendor counts disagree by month; the ranges above reflect Microsoft's own bulletin count versus third-party trackers (Tenable, BleepingComputer, N-able, Krebs on Security, Qualys, Lansweeper)
- Broader trackers that fold in republished third-party and Chromium-based Edge CVEs report far higher totals for the same months, e.g. ZDI counted 571 CVEs for June and 621 for July when Chromium fixes are included; Rapid7 counted 999 for September including 25 non-Microsoft CVEs
- September's release is the largest in Patch Tuesday history, surpassing July; AI-assisted vulnerability discovery is cited as a factor in the increased volume
- ZDI: 2,760 Microsoft CVEs patched so far in 2026 – more than double any previous full year

Microsoft Patch Tuesday – June 2026



- 198–208 CVEs, 32–37 critical, 3 zero-days, none exploited at release
- *CVE-2026-45586* – CTFMON EoP, public zero-day
- *CVE-2026-41089* – Netlogon RCE CVSS 9.8, disclosed May, confirmed exploited in June

Microsoft Patch Tuesday – July 2026



- 569–570 CVEs, 56–59 critical, 3 zero-days, 2 exploited
- *CVE-2026-56155* – AD FS EoP, exploited zero-day
- *CVE-2026-56164* – SharePoint EoP, exploited zero-day
- Largest Patch Tuesday release at the time, AI-assisted vulnerability discovery cited as a factor

Microsoft Patch Tuesday – August 2026



- 398–421 CVEs, 42–62 critical, 1 exploited zero-day
- *CVE-2026-68820* – afd.sys (WinSock) EoP, CVSS 7.0, on CISA KEV
<https://www.tenable.com/blog/microsofts-august-2026-patch-tuesday-addresses-398-cves-cve-2026-68820>
Exercise – check if any of these still affect your organizations
- Vendor counts disagree by month – Microsoft’s own bulletin count vs. third-party trackers

Microsoft Patch Tuesday – September 2026



- 964–974 CVEs, 50–113 critical, 2 exploited zero-days – largest Patch Tuesday ever
- *CVE-2026-81963* – Windows Update Stack EoP, CVSS 7.8, on CISA KEV
- *CVE-2026-85880* – Windows ALPC heap overflow EoP, CVSS 7.8, AppContainer sandbox escape, on CISA KEV
- *CVE-2026-69730* – Windows DNS Server RCE, use-after-free, CVSS 9.8, unauthenticated, not exploited at release
<https://www.tenable.com/blog/microsofts-september-2026-patch-tuesday-addresses-964-cves-cve-2026-81963-cve-2026-85880-cve-2026-69730>
- Out-of-band update needed days later: the patches broke Remote Desktop Services, Hyper-V Linux VMs and USB audio

Not just Microsoft

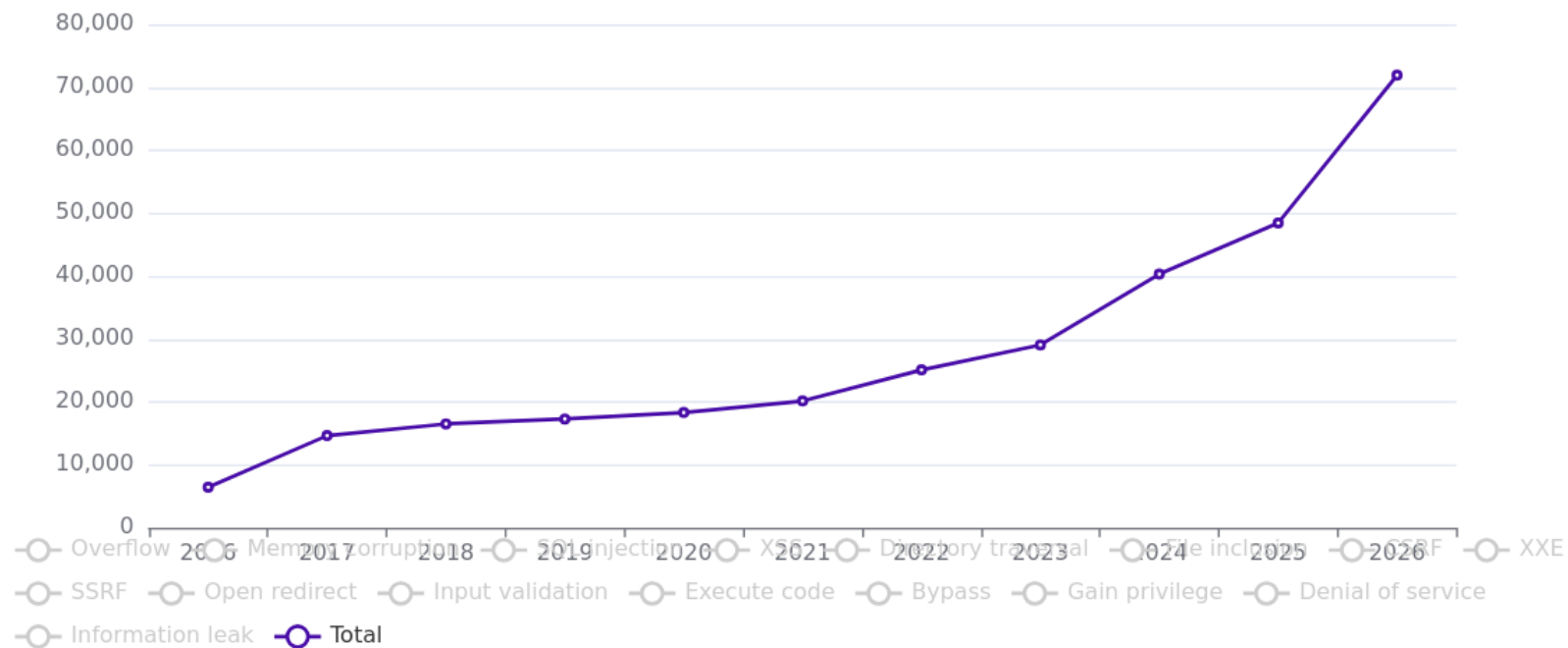


- Microsoft is a well-known vendor here, but far from the only vendor with high numbers
- *Linux kernel* – 5,681 CVEs in 2025 (highest single year ever recorded); already 2,308 CVEs in H1 2026, the single highest vendor count for the period
<https://linuxiac.com/linux-tops-2026-cve-charts/>
- *Fortinet* – 27 new CVEs disclosed in a single two-day stretch in April 2026 across FortiOS, FortiSandbox, FortiWeb and more, several critical/unauthenticated
<https://www.greenbone.net/en/blog/fortinet-rce-vulnerabilities-2026-critical-vulnerabilities-in-fortisandbox/>
- Other vendors worth showing: *Oracle* (1,235 CVEs in a single Critical Patch Update, July 2026), *Google/Chromium* (1,752 CVEs H1 2026), *Adobe*, *Apache*

Vulnerabilities by type & year



Vulnerabilities by type & year



Data from <https://www.cvedetails.com/> but see also main link: <https://www.cve.org/>

Exercise: browse CVE sites, find vulnerabilities in software you use



Try visiting the sites:

- CVE Main site <https://www.cve.org>
- Nice searching and graphs <https://www.cvedetails.com>
- Even more graphs and insights <https://zerodayclock.com/>

Please spend a little time, try searching for some vendor you know

Bookmark the sites for future reference.

Session 1: How does *bad software* happen?!



- How does *bad software* happen?!
- Assumptions and dependencies
- OWASP Top 10, CVE/CWE and Monster mitigations
- OWASP Juice Shop running on Kubernetes
<https://codeberg.org/kramse/kramse-labs>
- Exercise: look up OWASP Juice Shop and the companion guide

Assumptions



Any security policy, mechanism, or procedure is based on assumptions that, if incorrect, destroy the super-structure on which it is built.

Matt Bishop, Computer Security 2019

Example, vendor patches

Important points:

- Is the patch correct? Example Spectre and Heartbleed
- Vendor test environments equal to intended environments
- Installed correctly - including operator skills

Overview of Software Security



- Security Testing Versus Traditional Software Testing
- Functional testing does not prevent security issues!
- SQL Injection example, injecting commands into database
- Attackers try to break the application, server, operating system, etc.
- Use methods like user input, memory corruption / buffer overflow, poor exception handling, broken authentication, ...

Where to start?



The OWASP Top Ten provides a minimum standard for web application security. The OWASP Top Ten represents a broad consensus about what the most critical web application security flaws are.

The Open Worldwide Application Security Project (OWASP), previously Open Web Application Security Project

OWASP produces lists of the most common types of errors in web applications

<https://owasp.org>

Create Secure Software Development Lifecycle

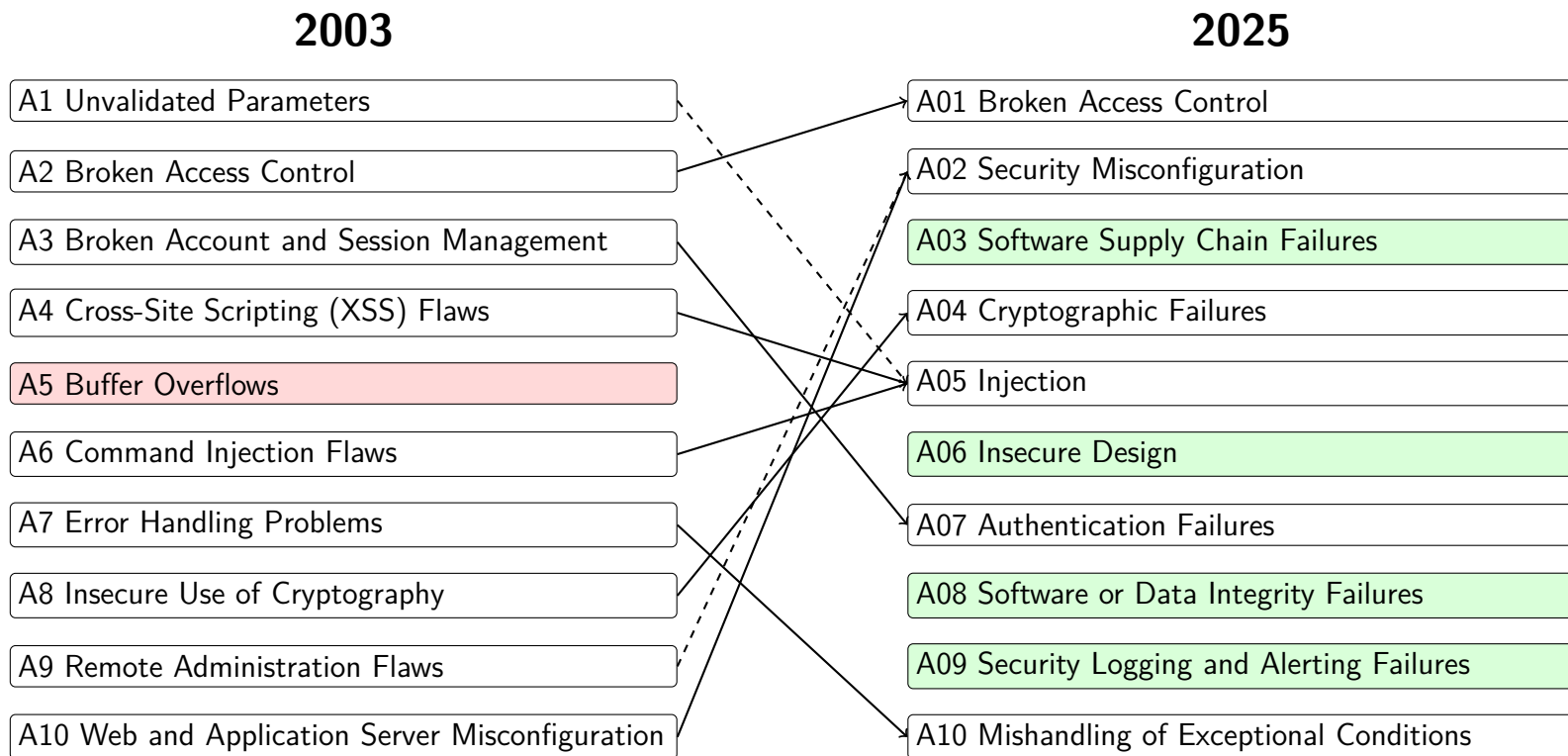
OWASP Top 10:2025



- A01:2025 - Broken Access Control
- A02:2025 - Security Misconfiguration
- A03:2025 - Software Supply Chain Failures
- A04:2025 - Cryptographic Failures
- A05:2025 - Injection – everything from XSS to SQL injection
- A06:2025 - Insecure Design
- A07:2025 - Authentication Failures
- A08:2025 - Software or Data Integrity Failures
- A09:2025 - Security Logging and Alerting Failures
- A10:2025 - Mishandling of Exceptional Conditions

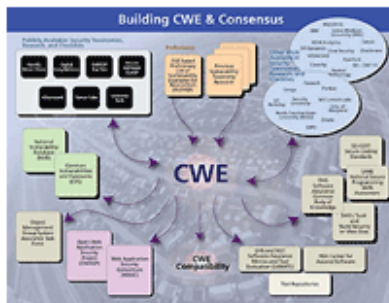
Source: <https://owasp.org/Top10/2025/>

OWASP Top 10: first (2003) vs latest (2025)



Red: no longer on the list. Green: no 2003 counterpart. Dashed: loose mapping. <https://owasp.org/Top10/2025/>

CWE Common Weakness Enumeration



[Enlarge](#)

CWE™ International in scope and free for public use, CWE provides a unified, measurable set of software weaknesses that is enabling more effective discussion, description, selection, and use of software security tools and services that can find these weaknesses in source code and operational systems as well as better understanding and management of software weaknesses related to architecture and design.

CWE in the Enterprise

- ▲ [Software Assurance](#)
- ▲ [Application Security](#)
- ▲ [Supply Chain Risk Management](#)
- ▲ [System Assessment](#)
- ▲ [Training](#)
- ▲ [Code Analysis](#)
- ▲ [Remediation & Mitigation](#)
- ▲ [NVD \(National Vulnerability Database\)](#)
- ▲ [Recommendation ITU-T X.1524 CWE, ITU-T CYBEX Series](#)

<https://cwe.mitre.org/>

CWE/SANS Monster mitigations



Monster Mitigations

These mitigations will be effective in eliminating or reducing the severity of the Top 25. These mitigations will also address many weaknesses that are not even on the Top 25. If you adopt these mitigations, you are well on your way to making more secure software.

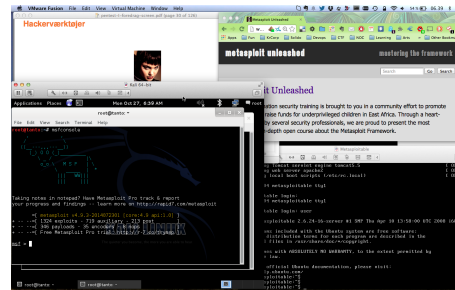
A [Monster Mitigation Matrix](#) is also available to show how these mitigations apply to weaknesses in the Top 25.

ID	Description
M1	Establish and maintain control over all of your inputs.
M2	Establish and maintain control over all of your outputs.
M3	Lock down your environment.
M4	Assume that external components can be subverted, and your code can be read by anyone.
M5	Use industry-accepted security features instead of inventing your own.
GP1	(general) Use libraries and frameworks that make it easier to avoid introducing weaknesses.
GP2	(general) Integrate security into the entire software development lifecycle.
GP3	(general) Use a broad mix of methods to comprehensively find and prevent weaknesses.
GP4	(general) Allow locked-down clients to interact with your software.

See the [Monster Mitigation Matrix](#) that maps these mitigations to Top 25 weaknesses.

Source: <https://cwe.mitre.org/top25/index.html>

Kubernetes hacker lab setup



- Container and Virtualisation software: pick your own choice
- Recommended: Rancher Desktop with Kubernetes as the vulnerable systems
- Hacker software: maybe a nearby Kali as a Virtual Machine <https://www.kali.org/>
- Smallest Kubernetes: Minikube - I have run this many times
- Production deployment probably would use better tools like  Kubectl
- Some instructions can be found at <https://codeberg.org/kramse/kramse-labs>

OWASP Juice Shop Project



We will also use the OWASP Juice Shop Tool Project as a running example. This is an application which is modern AND designed to have security flaws.

Read more about this project at: <https://owasp.org/www-project-juice-shop/>
<https://github.com/juice-shop/juice-shop>

Companion guide *Pwning OWASP Juice Shop* <https://pwning.owasp-juice.shop>

Why Kubernetes / k3s?



Run K8s Anywhere

Kubernetes is open source giving you the freedom to take advantage of on-premises, hybrid, or public cloud infrastructure, letting you effortlessly move workloads to where it matters to you.

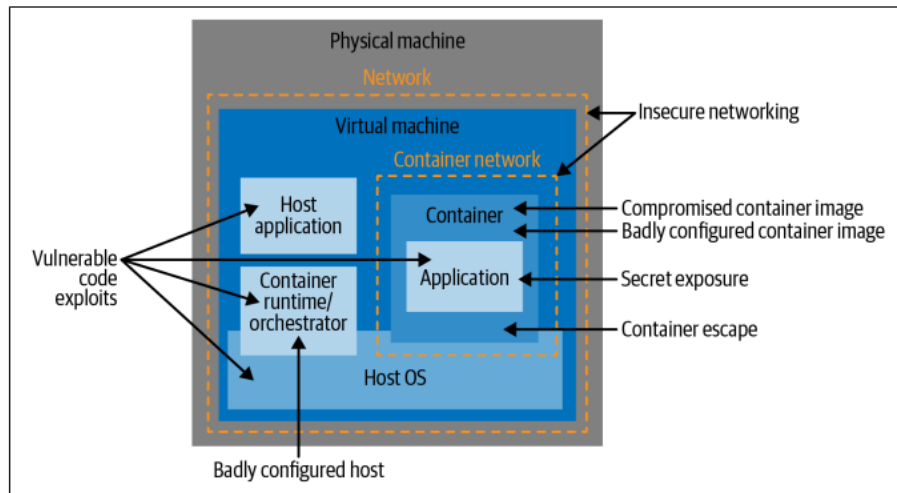
To download Kubernetes, visit the [download](#) section.

We often use a small local Kubernetes cluster (k3s, via Rancher Desktop) as the runtime for exercises. It gives everyone the same container-based exercise environment regardless of laptop OS (Windows, Mac, Linux).

- Faster to start, stop, and reset an exercise than a VM
- Same image runs identically on your laptop as it would in production
- Closer to how real-world applications are actually deployed and attacked today

Read more at: <https://k3s.io/> and <https://kubernetes.io/>

Kubernetes Security is more than just the core



Source: *Container Security*, (CS) Liz Rice, O'Reilly, Apr 2020

- Attacks from inside, attacks inside containers
- Some just use up resources, some attack others, some attack the system from the inside
- *Hacking Kubernetes* has a whole *Chapter 3 Container Runtime Isolation*

Exercise: look up OWASP Juice Shop



Try visiting the sites:

- Project main site <https://owasp-juice.shop>
- Source code <https://github.com/juice-shop/juice-shop>
- Companion guide *Pwning OWASP Juice Shop* <https://pwning.owasp-juice.shop>
- Running Juice Shop on Kubernetes <https://codeberg.org/kramse/kramse-labs>

Please spend a little time, look at the challenges in the companion guide and how they relate to the OWASP Top 10

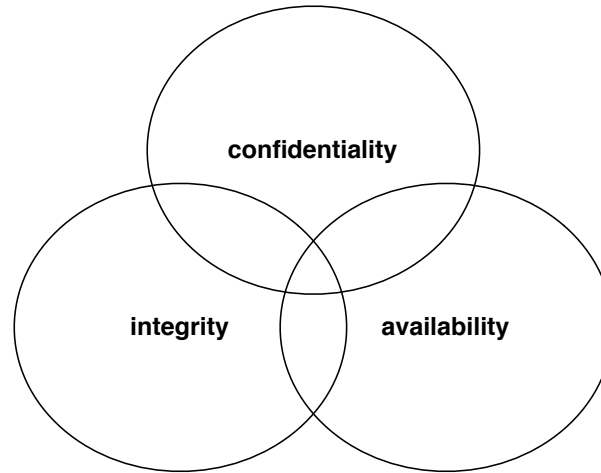
Bookmark the sites for future reference.

Session 2: Security principles and design



- How to reach the goal of secure design
- Security Principles for software
- Principle of least privilege, fail-safe defaults, separation of privilege etc.
- Security patterns, insecure and secure design
- Exercise: least privilege – everything runs as root

Confidentiality, Integrity and Availability



We want to protect something

Confidentiality - data kept a secret

Integrity - data is not subjected to unauthorized changes

Availability - data and systems are available when needed

Security is a process



Remember:

- what is information and security?
- Data kept electronically
- Data kept in physical form
- Don't forget the human element of security

Incident Response and Computer Forensics reaction to incidents

Good security is the result of planning and long-term work

Security is a process, not a product, Bruce Schneier

Source for quote: https://www.schneier.com/essays/archives/2000/04/the__process_of__secur.html

Work together



Team up!

We need to share security information freely

We often face the same threats, so we can work on solving these together

Goals of Security

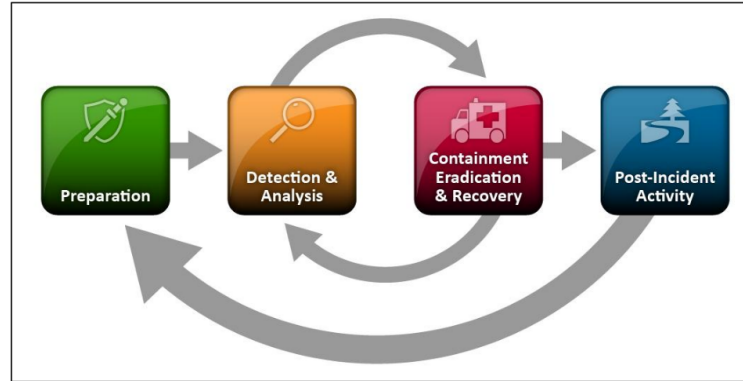


Figure 3-1. Incident Response Life Cycle

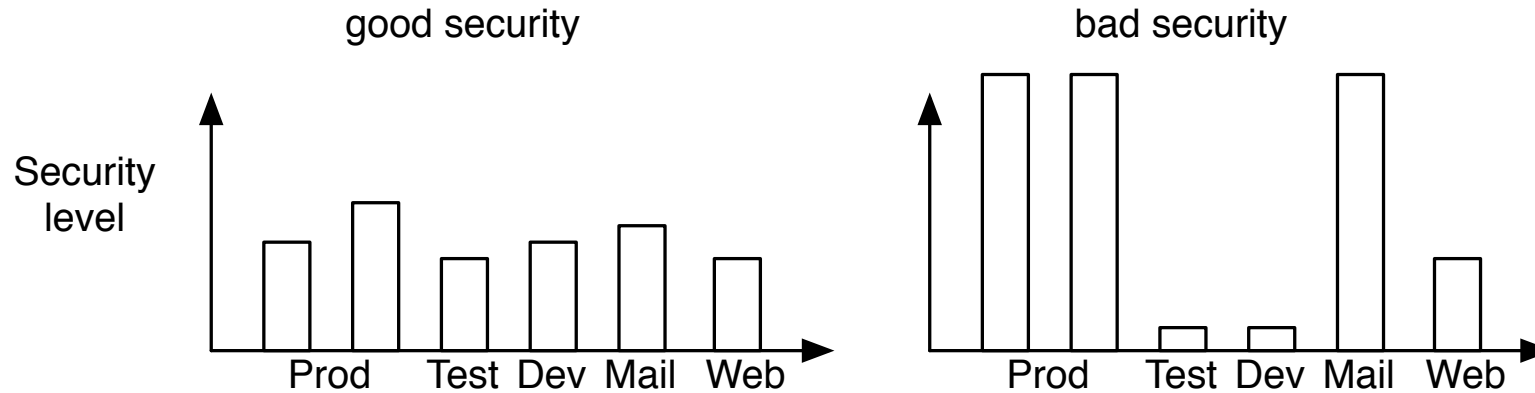
Source: *Computer Security Incident Handling Guide*, NIST SP 800-61 Rev. 2

Prevention - means that an attack will fail

Detection - determine if an attack is underway, or has occurred - report it

Recovery - stop attack, assess damage, repair damage

Balanced security



Better to have the same level of security

If you have bad security in some part - guess where attackers will end up

Hackers are not required to take the hardest path into the network

Realize there is no such thing as 100% security

Weak Structural Security



Example design flaws:

- Large Attack surface
- Running a Process at Too High a Privilege Level, don't run everything as root or administrator
- No Defense in Depth, use more controls, make a strong chain
- Not Failing Securely
- Mixing Code and Data
- Misplaced trust in External Systems
- Insecure Defaults
- Missing Audit Logs

Security Principles



We will use these as a reference:

- The Protection of Information in Computer Systems is a 1975 seminal publication by Jerome Saltzer and Michael Schroeder about information security. The paper emphasized that the primary concern of security measures should be the information on computers and not the computers themselves.
https://en.wikipedia.org/wiki/The_Protection_of_Information_in_Computer_Systems
- Principle of least privilege – individual principles often have their own pages:
https://en.wikipedia.org/wiki/Principle_of_least_privilege
- See also *Security by Design Principles*
Selected as OWASP has existed for many years, expect site to survive
https://www.owasp.org/index.php/Security_by_Design_Principles

Principle of Least Privilege



The *principle of least privilege* states that a subject should be given only those privileges that it needs in order to complete the task.

Source: *Computer Security*, Matt Bishop, 2019

- Originally formulated by Saltzer and Schroeder in 1975: every program and every user should operate with only the privileges needed for the job
- Also drop privileges when not needed anymore, relinquish rights immediately
- Example, need to read a document - but not write.
- Database systems can often provide very fine-grained access to data

Principle of Fail-Safe defaults



The *principle of fail-safe defaults* states that, unless a subject is given explicit access to an object, it should be denied access to that object.

Source: *Computer Security*, Matt Bishop, 2019

- Saltzer and Schroeder 1975: “Base access decisions on permission rather than exclusion”
- Default access *none*
- In firewalls default-deny - that which is not allowed is prohibited
- Newer devices today can come with no administrative users, while older devices often came with default admin/admin users
- Real world example, OpenSSH `PermitRootLogin` – root login with password is disabled by default

Fail securely



```
isAdmin = true;
try {
    codeWhichMayFail();
    isAdmin = isUserInRole( "Administrator" );
}
catch (Exception ex) {
    log.write(ex.toString());
}
```

“If either `codeWhichMayFail()` or `isUserInRole` fails or throws an exception, the user is an admin by default. This is obviously a security risk.”

Source: OWASP Security by Design Principles

https://www.owasp.org/index.php/Security_by_Design_Principles

- DO use exception handling!
- Just make sure code does not end up giving more privileges when failing!

Separation of Duties and Function



Separation of duties (SoD; also known as Segregation of Duties) is the concept of having more than one person required to complete a task. In business the separation by sharing of more than one individual in one single task is an internal control intended to prevent fraud and error.

Quote from https://en.wikipedia.org/wiki/Separation_of_duties

Separation of function. Developers do not develop new programs on production systems because of the potential threat to production data.

Computer Security, Matt Bishop, 2019

Danish: Funktionsadskillelse

Role-based Access Control (RBAC)



In computer systems security, **role-based access control (RBAC)**[1][2] or role-based security[3] is an approach to restricting system access to unauthorized users. It is used by the majority of enterprises with more than 500 employees,[4] and can implement mandatory access control (MAC) or discretionary access control (DAC).

Role-based access control (RBAC) is a policy-neutral access-control mechanism defined around **roles and privileges**. The components of RBAC such as role-permissions, user-role and role-role relationships make it simple to perform user assignments. A study by NIST has demonstrated that RBAC addresses many needs of commercial and government organizations[citation needed]. RBAC can be used to facilitate administration of security in large organizations with hundreds of users and thousands of permissions. Although RBAC is different from MAC and DAC access control frameworks, it can enforce these policies without any complication.

Quote from https://en.wikipedia.org/wiki/Role-based_access_control

Example role based system: PostgreSQL security



	11	10	9.6	9.5	9.4
Channel binding for SCRAM authentication	Yes	No	No	No	No
Column level permissions	Yes	Yes	Yes	Yes	Yes
Default permissions	Yes	Yes	Yes	Yes	Yes
GRANT/REVOKE ON ALL TABLES/SEQUENCES/FUNCTIONS	Yes	Yes	Yes	Yes	Yes
GSSAPI support	Yes	Yes	Yes	Yes	Yes
Large object access controls	Yes	Yes	Yes	Yes	Yes
Native LDAP authentication	Yes	Yes	Yes	Yes	Yes
Native RADIUS authentication	Yes	Yes	Yes	Yes	Yes
Per user/database connection limits	Yes	Yes	Yes	Yes	Yes
ROLES	Yes	Yes	Yes	Yes	Yes
Row-Level Security	Yes	Yes	Yes	Yes	No
SCRAM-SHA-256 Authentication	Yes	Yes	No	No	No
Search+bind mode operation for LDAP authentication	Yes	Yes	Yes	Yes	Yes
security_barrier option on views	Yes	Yes	Yes	Yes	Yes
Security Service Provider Interface (SSPI)	Yes	Yes	Yes	Yes	Yes
SSL certificate validation in libpq	Yes	Yes	Yes	Yes	Yes
SSL client certificate authentication	Yes	Yes	Yes	Yes	Yes
SSPI authentication via GSSAPI	Yes	Yes	Yes	Yes	Yes

Feature overview security features in PostgreSQL

<https://www.postgresql.org/about/featurematrix/#security>



All the principles today lead to **Security by Design**

- Secure defaults, minimize attack surface, fail securely
- Least privilege – no more “everything runs as root”
- Security decisions are explicit choices, not accidents

Applications don't run in a vacuum

- They run in existing environments – integrate with existing solutions, don't expect a clean slate
- Make it easy to use by following conventions – comply or explain why not

Security from requirements and design to deployment

- Threat modeling during design, checklists and cheat sheets to avoid repeating mistakes

Source: OWASP Developer Guide, 2.3 Principles of security and 4 Design

<https://owasp.org/www-project-developer-guide/release/>

See also: ENISA Secure by Design and Default Playbook

Breaking News: ENISA Secure by Design and Default Playbook



Hot off the press! – well, almost: published 30 July 2026

ENISA Secure by Design and Default Playbook

A Practical Guide to Secure by Design and Default Principles for SMEs

- Principles turned into clear, repeatable actions
- Fits into existing engineering, product and release processes
- Covers the whole product life cycle
- 22 playbooks, browsable on GitHub

<https://www.enisa.europa.eu/publications/enisa-secure-by-design-and-default-playbook>

<https://github.com/enisaeu/enisa-sbd-playbook/>

Today's principles are not old news – the EU is still writing about them!

Stay Tuned: Where Security News Comes From



The field moves – the slides you see today will be outdated tomorrow

Follow the sources, not just the headlines:

- OWASP – Top 10, cheat sheets, Juice Shop

<https://owasp.org>

- ENISA – EU Agency for Cybersecurity, Cyber Resilience Act guidance

<https://www.enisa.europa.eu/publications>

- NIST – Secure Software Development Framework (SSDF), SP 800-218

<https://csrc.nist.gov/projects/ssdf>

ENISA has an e-mail alert service – sign up once, get the news delivered

<https://www.enisa.europa.eu/alertservice>

Exercise: least privilege – everything runs as root



Two cases – select the one that speaks most to you:

ERNW White Paper 68, Security Assessment of Cisco ACI

<https://ernw.de/en/whitepapers/issue-68.html>

- 2.1 Remote Code Execution on Leaf Switches over IPv6 via Local SSH Server (CVE-2019-1836, CVE-2019-1803, and CVE-2019-1804)
- 2.2.2 Cisco Nexus 9000 Series Fabric Switches Application Centric Infrastructure Mode Link Layer Discovery Protocol Buffer Overflow Vulnerability (CVE-2019-1901)
- 2.3 Cisco Application Policy Infrastructure Controller REST API Privilege Escalation Vulnerability (CVE-2019-1889)

Cisco Secure Email Gateway SQL Injection, CVE-2026-76461

<https://www.cisco.com/c/en/us/support/docs/csa/cisco-sa-esa-inj-2bLVGmhX.html>

Imagine you are the developer receiving this information: Where is least privilege violated? Which CWE is it? How serious would the vulnerability have been without root privileges?

Session 3: Code auditing, analysis and security testing



- Security Testing Versus Traditional Software Testing
- Code-Auditing Strategies: static analysis and dynamic analysis
- Exercise: clone OWASP Juice Shop and find the SQL injection using grep or project search in your editor

Testing - more work now, less work in the long run



	Test1
	Test2
	Test3
	Test4

Unit testing - low level / functions



```
public class TestAdder {  
    public void testSum() {  
        Adder adder = new AdderImpl();  
        assert(adder.add(1, 1) == 2);  
        assert(adder.add(1, 2) == 3);  
        assert(adder.add(2, 2) == 4);  
        assert(adder.add(0, 0) == 0);  
        assert(adder.add(-1, -2) == -3);  
        assert(adder.add(-1, 1) == 0);  
        assert(adder.add(1234, 988) == 2222);  
    }  
}
```

Test individual functions

Example from https://en.wikipedia.org/wiki/Unit_testing

Avoid regressions, old errors reappearing



```
main(int argc, char **argv)
{
    char buf[200];
    strcpy(buf, argv[1]);
    printf("%s\n", buf);
}
```

Use tools for analysing code and applications

Hard to do – manual analysis



Why not just program securely?

There are many resources available:

Websites: *Secure Programming for Linux and Unix HOWTO*

<https://dwheeler.com/secure-programs/>

Books: *24 Deadly Sins of Software Security: Programming Flaws and How to Fix Them*

Michael Howard, David LeBlanc, John Viega + their other books

It is too hard, too expensive, takes too long!?

Why can't we just read the code?



- Code bases are large – and most of the code is not yours: frameworks, libraries, dependencies
- Code changes every day – a review is outdated with the next commit
- Humans get tired and miss patterns, tools do not
- Vulnerabilities often hide in the interaction between components, not in a single line
- Manual review is still valuable – tools find the known patterns, humans find the logic and design errors

Types of analysis



static analysis

finds errors without running the program

typically finds constructs containing errors, use of wrong functions etc.

also called SAST – Static Application Security Testing

dynamic analysis

finds errors by running the program, typically in a special environment

includes unit testing, debugging and code coverage – in security also fuzzing

also called DAST – Dynamic Application Security Testing

Static analysis tools – SAST



- Semgrep, many languages, rule based <https://semgrep.dev>
- Bandit, Python security issues <https://github.com/PyCQA/bandit>
- Ruff, Python linter including Bandit rules <https://docs.astral.sh/ruff/>
- Flawfinder, C and C++ <https://dwheeler.com/flawfinder/>
- PMD, Java and other languages <https://pmd.github.io>

Many more exist https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis

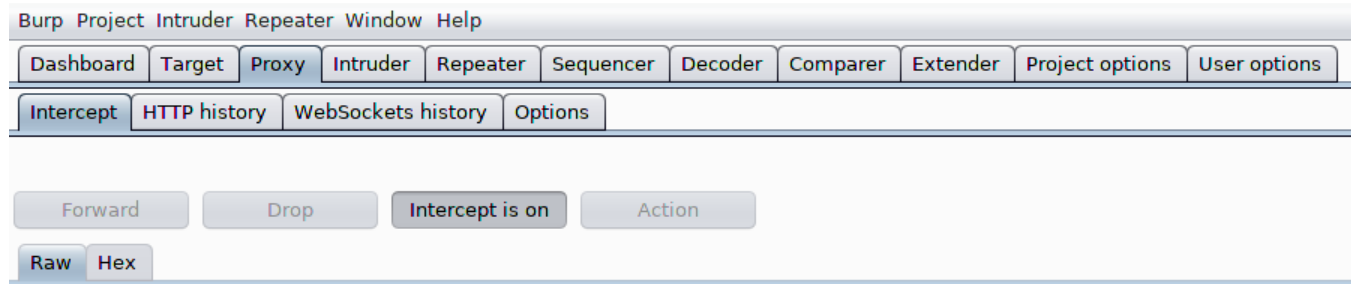
Dynamic analysis for developers – run it in a harness



- Sanitizers in GCC and Clang: AddressSanitizer, UndefinedBehaviorSanitizer
compile with `-fsanitize=address,undefined` – the program aborts on memory errors
<https://github.com/google/sanitizers>
- Valgrind, memory errors and leaks at runtime, no recompile needed
<https://valgrind.org>
- AFL++ and libFuzzer, coverage-guided fuzzing – feed random input, look for crashes
<https://aflplus.plus> <https://llvm.org/docs/LibFuzzer.html>
- Atheris, coverage-guided fuzzing for Python
<https://github.com/google/atheris>
- Hypothesis, property-based testing for Python – generates test input for your unit tests
<https://hypothesis.works>
- OSS-Fuzz, continuous fuzzing of open source projects
<https://google.github.io/oss-fuzz/>

Sorry, I DO NOT use these regularly, some I haven't tried myself!

Dynamic analysis tools – DAST, testing the running application



- Burp, web application scanner and proxy <https://portswigger.net/burp>
- Nikto, web server scanner <https://github.com/sullo/nikto>
- sqlmap, SQL injection testing <https://sqlmap.org>

A Fool with a Tool is still a Fool



1. Run tool
2. Fix problems
3. Rinse repeat

Fixing problems?

```
char tmp[256]; /* Flawfinder: ignore */  
strcpy(tmp, pScreenSize); /* Flawfinder: ignore */
```

Example from <https://dwheeler.com/flawfinder/>

Fixing the warning – Debian OpenSSL 2006–2008



- 2006: a Debian developer removes code from OpenSSL to silence Valgrind warnings about use of uninitialized memory
- The removed code fed entropy into the random number generator
- Result: the only randomness left was the process ID
 - at most 32,768 different keys per key type and size
- May 2008: discovered by Luciano Bello, CVE-2008-0166, DSA-1571-1
- Affected SSH keys, OpenVPN keys, DNSSEC keys, X.509 certificates, TLS session keys
- Worldwide regeneration of keys – and weak keys were still found in use years later
- The tool was right to warn – the fix was wrong

More at: https://en.wikipedia.org/wiki/Random_number_generator_attack#Debian_OpenSSL

The random number generator is VITAL for crypto security

Exim RCE CVE-2019-10149 June



VULNERABILITY PATCHED... BY ACCIDENT ...

This was only recently discovered by the Qualys team while auditing older Exim versions. Now, Qualys researchers are warning Exim users to update to the 4.92 version to avoid having their servers taken over by attackers. Per the same June 2019 report on email server market share, only 4.34% of all Exim servers run the latest 4.92 release.

In an email to Linux distro maintainers, Qualys said the vulnerability is "trivially exploitable" and expects attackers to come up with exploit code in the coming days.

This Exim flaw is currently tracked under the CVE-2019-10149 identifier, but Qualys refers to it under the name of **"Return of the WIZard"** because the **vulnerability resembles the ancient WIZ and DEBUG vulnerabilities that impacted the Sendmail email server back in the 90s.**

<https://www.zdnet.com/article/new-rce-vulnerability-impacts-nearly-half-of-the-internets-email-servers/>

See also detailed information from the finders:

<https://www.qualys.com/2019/06/05/cve-2019-10149/return-wizard-rce-exim.txt>

Exim RCE CVE-2019-15846 September



The Exim mail transfer agent (MTA) software is impacted by a critical severity vulnerability present in versions 4.80 up to and including 4.92.1.

The bug allows local or unauthenticated remote attackers to execute programs with root privileges on servers that accept TLS connections.

The flaw tracked as CVE-2019-15846 — initially reported by 'Zerons' on July 21 and analyzed by Qualys' research team — is **"exploitable by sending an SNI ending in a backslash-null sequence during the initial TLS handshake"** which leads to **RCE with root privileges on the mail server.**

<https://www.bleepingcomputer.com/news/security/critical-exim-tls-flaw-lets-attackers-remotely-execute-commands-as-root/>

Exercise: find the SQL injection in OWASP Juice Shop



The Juice Shop login is vulnerable to SQL injection – find the code!

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email} || '''`  
AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`,  
{ model: models.User, plain: true })
```

- Clone the source code and search for the database lookups
- Use a simple tool: grep, or project search in your editor
- Tip: search for SELECT * FROM and filter out tests and frontend

```
git clone https://github.com/juice-shop/juice-shop.git  
cd juice-shop;grep -ril SELECT | egrep -v "test|frontend|static"
```

Discussion: how much would it take to change this into prepared statements?

Session 4: Coding standards, Git hooks and scanner tools



- Policies, coding standards, compiler warnings, defensive programming
- Git hooks and scanner integration in continuous integration
- Tools: Semgrep, Bandit, Ruff
- Exercise: research Semgrep as Git hook and CI tool
<https://semgrep.dev/docs/extensions/pre-commit>
Discuss: which tools does your team run today – locally, in hooks or in CI?

Introduction to Methods



- The following slides list some of the methods for eliminating security issues
- When eliminating them is not possible, the runtime environment can sometimes be improved
- Other times the strategy involves the systems used for development – like using the version control system to enforce policies and prevent developers from doing bad things

Higher quality software is often more secure

Hello World



While small test programs have existed since the development of programmable computers, the tradition of using the phrase "Hello, World!" as a test message was influenced by an example program in the seminal 1978 book *The C Programming Language*.^[3] The example program in that book prints "hello, world", and was inherited from a 1974 Bell Laboratories internal memorandum by Brian Kernighan, *Programming in C: A Tutorial*.^[4]

```
main( ) {  
    printf("hello, world\n");  
}
```

- Very compact, easily readable - more than assembler anyway!

Code and quote from https://en.wikipedia.org/wiki/%22Hello,_World!%22_program

Hello World



```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    (void) argc;
    (void) argv;

    printf("Hello world!\n");

    return EXIT_SUCCESS;
}
```

- Don't forget to define function return value, your variables and arguments to the program and functions
- Don't forget to return something useful
- What's an int anyway? integers

Source: a better hello world from https://da.wikipedia.org/wiki/Hello_world-program#C

Data types



- Character types char, signed char, unsigned char
- Integer types, short int, int, long int, long long int - plus unsigned
- Floating types float, double, long double
- Bit fields a specific number of bits in an object
- Signed and unsigned!
- Byte order, little or big endian - named after the 1726 novel Gulliver's Travels by Jonathan Swift
- Big endian also called network order, as used in TCP/IP suite
- Lots more types we won't discuss in detail

Data types have a *range of values*

See more details at https://en.wikipedia.org/wiki/C_data_types

also interesting is the float format

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

Example integer overflow



```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv)
{
    (void) argc; (void) argv;
    short int i1 = 32767;
    printf("First debug int is %d\n", i1);
    i1++;
    printf("Second debug int is now %d \n", i1);
}
```

```
user@Projects:programs$ gcc -o int1 int1.c && ./int1
```

```
First debug int is 32767
```

```
Second debug int is now -32768
```

We won't do the math in binary!

Overriding types, type conversion



Unsigned 00000001 is the same as 000000000000000001

- Value-preserving - there is space in the new type to store all values
- Value-changing - new type cannot represent the same values, damn!
- To a wider type, we can *sign extend* for signed types and *zero extend* for unsigned types
- To a narrower type, we need to truncate – bye bye bits, lose precision!
- Again, we won't repeat all the bit values and arithmetic, and how C extends the values
- Casting can be used to specify the explicit type conversion
(unsigned char) bob - treat variable bob as an unsigned char type

Audit tip: The compiler can optimize out certain conversions! This can affect both cryptographic processes and other things! Beware!

Type Conversion Vulnerabilities



```
int copy(char *dst, char *src, unsigned int len)
{
    while (len--)
        *dst++ = *src++
}
```

- Signed/unsigned conversions
- If you pass a signed int to this, a conversion might result in a large value instead
- This large value will most likely result in overflow
- Most libc routines that take a size have `size_t` which is unsigned!
- Signed negative values become very large unsigned

Coding standards - style



This file specifies the preferred style for kernel source files in the OpenBSD source tree. It is also a guide for preferred user land code style. These guidelines should be followed for all new code. In general, code can be considered “new code” when it makes up about 50% or more of the file(s) involved. ...

Use queue(3) macros rather than rolling your own lists, whenever possible. Thus, the previous example would be better written:

```
#include <sys/queue.h>
struct foo {
    LIST_ENTRY(foo) link; /* Queue macro glue for foo lists */
    struct mumble amumble; /* Comment for mumble */
    int bar;
};
LIST_HEAD(, foo) foohead; /* Head of global foo list */
```

OpenBSD style(9)

Coding standards functions



The following copies as many characters from input to buf as will fit and NUL terminates the result. Because strncpy() does not guarantee to NUL terminate the string itself, it must be done by hand.

```
char buf[BUFSIZ];  
  
(void)strncpy(buf, input, sizeof(buf) - 1);  
buf[sizeof(buf) - 1] = '\\0';
```

Note that strncpy(3) is a better choice for this kind of operation. The equivalent using strncpy(3) is simply:

```
(void)strncpy(buf, input, sizeof(buf));
```

OpenBSD strcpy(9)

Compiler warnings - gcc -Wall



```
# gcc -o demo demo.c
```

```
demo.c: In function main:
```

```
demo.c:4: warning: incompatible implicit declaration of built-in  
function strcpy
```

```
# gcc -Wall -o demo demo.c
```

```
demo.c:2: warning: return type defaults to int
```

```
demo.c: In function main:
```

```
demo.c:4: warning: implicit declaration of function strcpy
```

```
demo.c:4: warning: incompatible implicit declaration of built-in  
function strcpy
```

```
demo.c:5: warning: control reaches end of non-void function
```

Easy to do!

No warnings = no errors?



```
# cat demo2.c
#include <strings.h>
int main(int argc, char **argv)
{
    char buf[200];
    strcpy(buf, argv[1]);
    return 0;
}
# gcc -Wall -o demo2 demo2.c
```

This is an insecure program, but no warnings!

(cheating, some compilers actually warn today)

Suricata VXLAN decode - defensive checks



```
int DecodeVXLAN(ThreadVars *tv, DecodeThreadVars *dtv, Packet *p,
    const uint8_t *pkt, uint32_t len, PacketQueue *pq)
{
    if (unlikely(!g_vxlan_enabled))
        return TM_ECODE_FAILED;
    if (len < (sizeof(VXLANHeader) + sizeof(EthernetHdr)))
        return TM_ECODE_FAILED;
    const VXLANHeader *vxlanh = (const VXLANHeader *)pkt;
    if ((vxlanh->flags[0] & 0x08) == 0 || vxlanh->res != 0) {
        return TM_ECODE_FAILED;
    }
    #if DEBUG
    uint32_t vni = (vxlanh->vni[0] << 16) + (vxlanh->vni[1] << 8) + (vxlanh->vni[2]);
    SCLogDebug("VXLAN vni %u", vni);
    #endif
    StatsIncr(tv, dtv->counter_vxlan);
}
```

Source: excerpt from <https://github.com/OISF/suricata/blob/master/src/decode-vxlan.c>

Don't read past the end of the buffer!

Version control sample hooks scripts



Before checking in code in version control, pre-commit - check

- case-insensitive.py
- check-mime-type.pl
- commit-access-control.pl
- commit-block-joke.py
- detect-merge-conflicts.sh
- enforcer
- log-police.py
- pre-commit-check.py
- verify-po.py

http://subversion.tigris.org/tools_contrib.html

<http://svn.collab.net/repos/svn/trunk/contrib/hook-scripts/>

This references Subversion, which is not used much anymore. Just to show the concept is NOT new. Use hooks!

Example Enforcer



In a Java project I work on, we use log4j extensively. Use of `System.out.println()` bypasses the control that we get from log4j, so we would like to discourage the addition of `println` calls in our code.

We want to deny any commits that add a `println` into the code. The world being full of exceptions, we do need a way to allow some uses of `println`, so we will allow it if the line of code that calls `println` ends in a comment that says it is ok:

```
System.out.println("No log4j here"); // (authorized)
```

<http://svn.collab.net/repos/svn/trunk/contrib/hook-scripts/enforcer/enforcer>

Example verify-po.py



```
#!/usr/bin/env python  
"""This is a pre-commit hook that checks whether the contents  
of PO files committed to the repository are encoded in UTF-8.  
"""
```

<http://svn.collab.net/repos/svn/trunk/tools/hook-scripts/verify-po.py>

Break it



Use fuzzers, hacker tools, improve security by breaking it

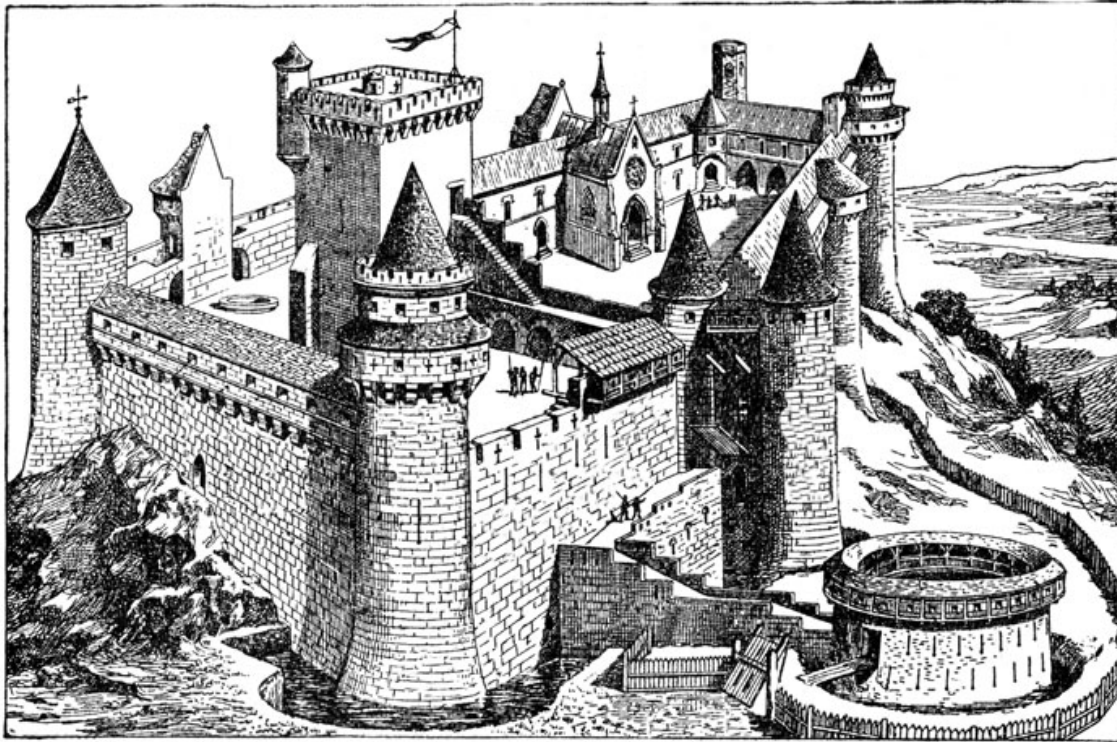
Simple fuzzer



```
$ for i in 10 20 30 40 50
>> do
>> ./demo `perl -e "print 'A'x$i"`
>> done
AAAAAAAAAA
AAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA
Memory fault
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Memory fault
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Memory fault
```

Memory fault/segmentation fault - juicy!

Enhance and secure runtime environment



Last chance when executing code! Crash is better than running malicious code!

Isolation: chroot, jails, containers and sandboxes



Many ways to isolate processes – all build on least privilege

- Unix chroot – changes the root directory, *not* a security boundary, still used inside daemons like OpenSSH
- FreeBSD Jails, OpenBSD `pledge(2)` and `unveil(2)` – the program restricts itself
- Linux capabilities – split root into small privileges, e.g. `CAP_NET_BIND_SERVICE`
- Linux namespaces and cgroups – the building blocks of containers: Docker, Podman, containerd
- seccomp – filter which system calls a process may use
- Mandatory access control: SELinux, AppArmor, Landlock
- Stronger isolation: gVisor, Kata Containers, Firecracker microVMs – and full virtual machines

Containers share the host kernel – unlike virtual machines, a kernel bug can break the isolation

JVM security policies



```
// ===== WEB APPLICATION PERMISSIONS =====
// These permissions are granted by default to all web applications
// In addition, a web application will be given a read FilePermission
// and JndiPermission for all files and directories in its document root.
grant {
    // Required for JNDI lookup of named JDBC DataSource's and
    // javamail named MimePart DataSource used to send mail
    permission java.util.PropertyPermission "java.home", "read";
    permission java.util.PropertyPermission "java.naming.*", "read";
    permission java.util.PropertyPermission "javax.sql.*", "read";
    ...
};
// The permission granted to your JDBC driver
// grant codeBase "jar:file:${catalina.home}/webapps/examples/WEB-INF/lib/driver.jar!/-" \{
//     permission java.net.SocketPermission "dbhost.mycompany.com:5432", "connect";
// \};
```

Example from apache-tomcat-6.0.18/conf/catalina.policy

Apple sandbox named generic rules



```
;; named - sandbox profile  
;; Copyright (c) 2006-2007 Apple Inc. All Rights reserved.  
;;  
;; WARNING: The sandbox rules in this file currently constitute  
;; Apple System Private Interface and are subject to change at any time and  
;; without notice. The contents of this file are also auto-generated and not  
;; user editable; it may be overwritten at any time.  
;;  
(version 1)  
(debug deny)  
  
(import "bsd.sb")  
  
(deny default)  
(allow process*)  
(deny signal)  
(allow sysctl-read)  
(allow network*)
```

Apple sandbox named specific rules



```
;; Allow named-specific files
(allow file-write* file-read-data file-read-metadata
  (regex "^(/private)?/var/run/named\\pid$"
    "^/Library/Logs/named\\.log$"))

(allow file-read-data file-read-metadata
  (regex "^(/private)?/etc/rndc\\.key$"
    "^(/private)?/etc/resolv\\.conf$"
    "^(/private)?/etc/named\\.conf$"
    "^(/private)?/var/named/"))
```

Example from /usr/share/sandbox on Mac OS X, old version

Isolation in practice – Kubernetes



Kubernetes Pod Security Standards, profile *restricted* – do not run containers as root

```
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: juice-shop
    image: bkimminich/juice-shop
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
```

Source: Kubernetes, Pod Security Standards <https://kubernetes.io/docs/concepts/security/pod-security-standards/>

Enforce it for a whole namespace: `kubectl label ns juice-shop pod-security.kubernetes.io/enforce=restricted`

Systemd service hardening



Named user, few rights – PostgreSQL on Debian runs as postgres

Add hardening without changing the package – `systemctl edit postgresql@17-main`

```
[Service]
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
ReadWritePaths=/var/lib/postgresql /var/run/postgresql /var/log/postgresql
CapabilityBoundingSet=
```

Check the result with `systemd-analyze security postgresql@17-main`

Same idea for Postfix as postfix, SSHD privilege separation as sshd

Container and isolation vulnerabilities – a timeline



... but no software is perfect

- 2016: Dirty COW, CVE-2016-5195 – Linux kernel, write to read-only memory
- 2019: runc, CVE-2019-5736 – overwrite the host runc binary from inside a container
- 2022: Dirty Pipe, CVE-2022-0847 – Linux kernel, overwrite read-only files
- 2024: Leaky Vessels, CVE-2024-21626 – runc, leaked file descriptor gives access to host filesystem
- 2025: runc, CVE-2025-31133, CVE-2025-52565, CVE-2025-52881 – container escape via mount races
- Kernel and container runtime bugs break the isolation – every few years, again and again
- Containers share the host kernel – patch the host, not just the images

runc November 2025 – patch and least privilege



Three container escape vulnerabilities in runc, disclosed November 2025

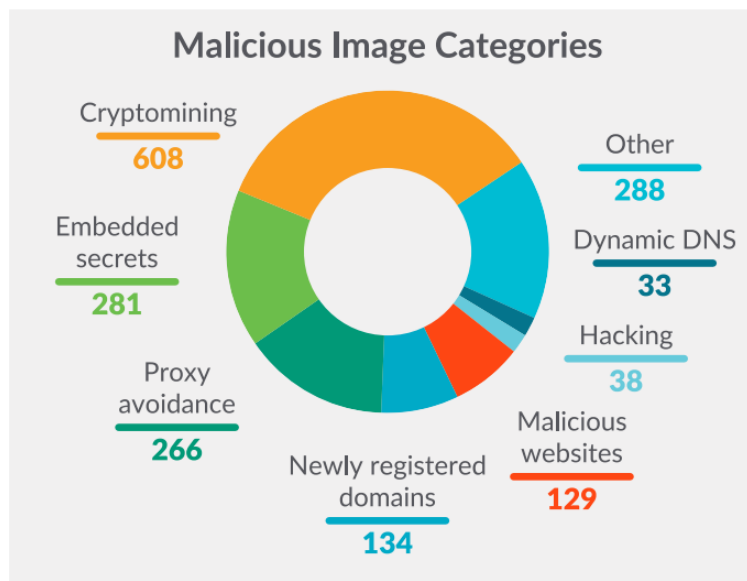
- runc is the runtime under Docker, containerd, CRI-O and Kubernetes
- A malicious or compromised container image can write to host /proc and escape
- CVE-2025-52881 is an incomplete fix for CVE-2019-16884 – old bugs come back

Mitigations:

- Update runc – fixed in 1.2.8, 1.3.3 and 1.4.0-rc.3
- Do not run containers as root, enable no new privileges
- Use user namespaces, so host root is not mapped into the container
- Avoid untrusted container images

<https://sysdig.com/blog/runc-container-escape-vulnerabilities>

Analysis on Docker Hub malicious images



This article is relevant, talking about malicious Docker images

<https://sysdig.com/blog/analysis-of-supply-chain-attacks-through-public-docker-images/>

Exercise: Semgrep as Git hook and CI tool



Part 1 – research, try visiting:

- Semgrep main site <https://semgrep.dev>
- Running Semgrep as a pre-commit hook
<https://semgrep.dev/docs/extensions/pre-commit>
- The pre-commit hooks for Semgrep
<https://github.com/semgrep/pre-commit>

Part 2 – discuss in smaller groups

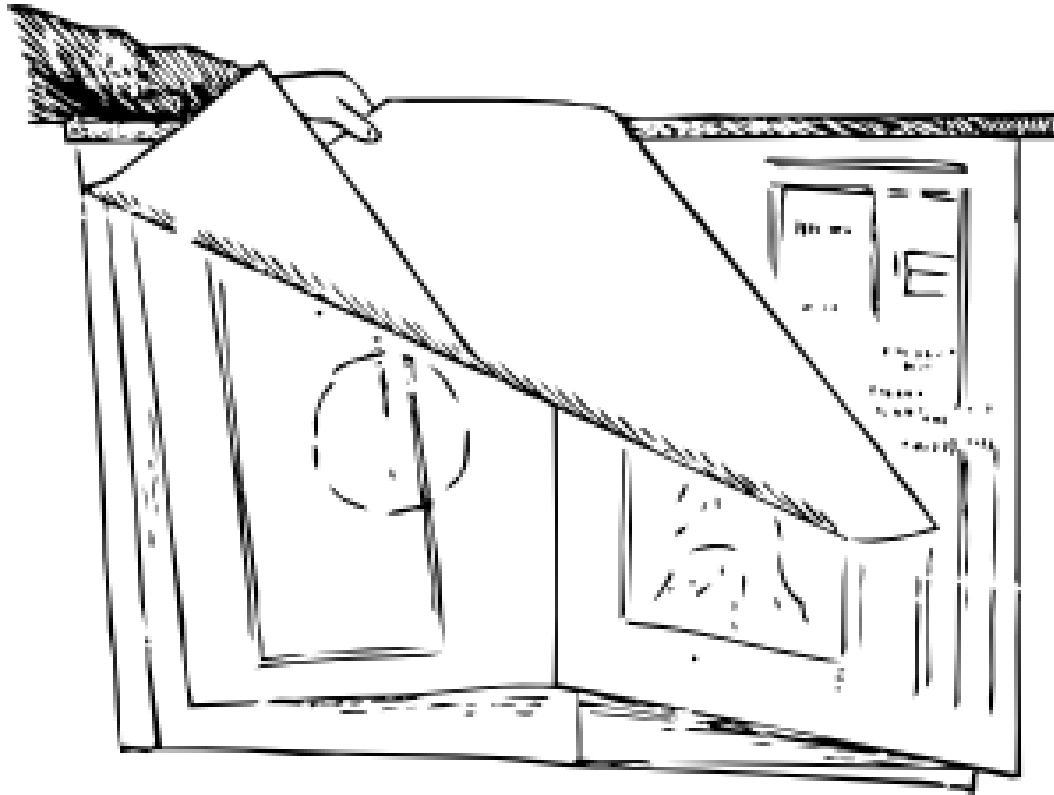
- Could Semgrep have stopped the Juice Shop SQL injection before the commit?
- Which tools does your team run today?
- Where do they run – locally, in Git hooks or in CI?
- What happens when a tool finds something – does the build fail?

Session 5: Secure Development Lifecycle and standards



- Secure Development Lifecycle (SDL, SDLC, SSDL) – multiple names, but essentially the same
- Standards ISO 27001/CIS Controls, Application Software Security
- ISO/IEC 27001:2022 Annex A 8.25–8.31
- Exercise: your processes and documentation vs. Annex A – where are the gaps?
Continues in the optional part at the end of the day

Design for security - more work



Security is cheapest and most effective when done during the design phase.

Secure Coding starts with the design



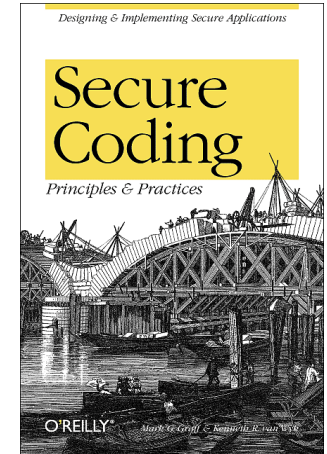
Secure Coding: Principles and Practices by Mark G. Graff, Kenneth R. Van Wyk 2003

Architecture/design – while you are thinking about the application

Implementation – while you are writing the application

Operations – after the application is in production

Ca. 200 pages, very dense.



Design a robust network Isolation and segmentation

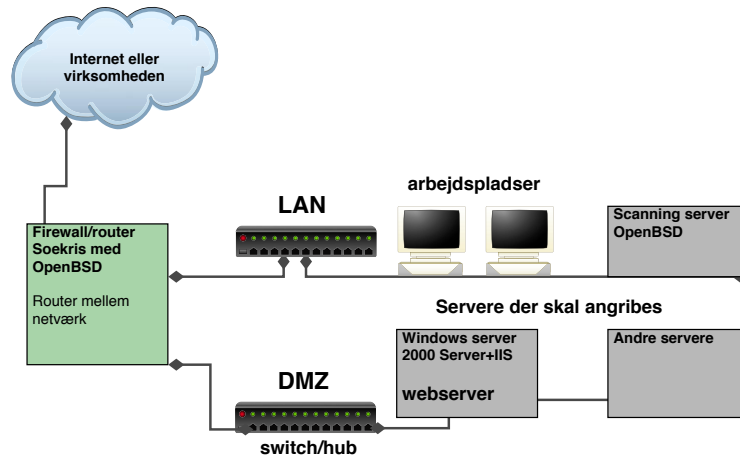


What can you do to improve network security?

- Use switches – sniffing then requires ARP spoofing, and better performance
- Divide with firewalls into multiple DMZ zones to keep exposed servers separated from each other, the internal network and the Internet
- Monitor, read logs and respond to incidents

Remember you also need to be able to update your servers

Basic Network Security Pattern: Isolate in VLANs



You should segment your network based on traffic

You should always keep internal and external systems separated!

You should isolate dangerous services in jails and chroots

Use port security to protect basic services like DHCP, Spanning Tree etc.

Building Secure Infrastructures



A real-life setup of an infrastructure from scratch can be daunting!

You need:

- Policies
- Procedures
- Incident Response

Running systems which require

- Configurations
- Settings
- Supporting infrastructure – networks
- Supporting infrastructure – logging, dashboarding, monitoring

Building something *secure* is **hard work!**

Existing infrastructures



Or even worse, you inherited an infrastructure

Multiple networks, with different vendors, rules

Multiple generations of services, applications, technologies

Built over decades

Varying to no documentation

Organizational challenges

Ingrained culture – frozen in time

How do you get started improving security?

Integrate or develop?



Don't:

- Reinvent the wheel - too many times, unless you can maintain it afterwards
- Invent cryptography yourself
- Copy-paste functionality, harder to maintain in the future

Do:

- Integrate with existing solutions
- Use existing well-tested code: cryptography, authentication, hashing
- Centralize security in your code and organization

What it means for Software Security

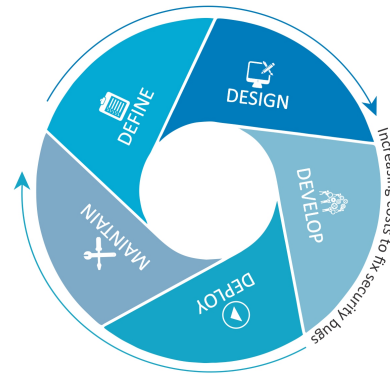


You need processes for working with system environments containing many servers – all with operating systems and applications

Attack the problems differently

- Test using penetration testing
- Read and scan source code
- Review designs
- Select frameworks and languages – policies

Secure Software Development Lifecycle



- SSDL is a structured approach to implementing and performing secure software development
- Security issues are evaluated and addressed early
- During business analysis – through the requirements phase – during design and implementation
- Picture from OWASP Testing Guide project, <https://owasp.org/www-project-web-security-testing-guide/>

Development Standards



What do you do to avoid problems like the ones mentioned here? – can you do more?

You should be aware of which technologies you use

Standardize on a smaller number of products, libraries, languages

Rules and procedures must be updated continuously:

- Quality assurance
- Guidelines for allowed tags
- Guidelines for use of SQL

By limiting the number of products you may reduce the opportunities for errors – high quality is often more secure

New products can be dangerous until you get to know them!

Guidelines



- If there are no guidelines for development, establish them
- Example:
JavaScript may be used to validate forms to give quick feedback to the user
- The server receiving input from the user validates all data for security
- Guidelines help make a balanced investment in security
- Avoid holes in security – keep a consistent level
- Guides exist for both old and new languages/systems,
e.g. Ruby on Rails Security Guide
<https://guides.rubyonrails.org/security.html>
- OWASP Cheat Sheet Series
<https://cheatsheetseries.owasp.org>

Preparing environments for production



Consider the environment that servers and services will be exposed to

Ensure hardening and think about general security:

- Updated software – no known security holes or vulnerabilities
- Remove **single points of failure** – redundant power, extra units, two DNS servers instead of one
- Separate servers – internal and external for different purposes
- Filter on the network or on data – firewalls and proxy functions
- Restrict access to read information
- Restrict access to write information – for example databases
- Use **least privilege** – make sure programs and users only have the rights needed to perform their tasks
- Keep up with the areas relevant to the company and *your* installation

Subscribe to security mailing lists for the products you use, also open source

Change management



Is there enough focus on software in production?

Can any server easily be re-established?

Are changes made directly on production systems?

Is there a fall-back plan?

This should be good system administration practice

The foundation must be in order



Make sure the infrastructure you build on is secure:

- redundant
- updated
- documented
- easy to maintain

Remember availability is also a security parameter

ISO/IEC 27001:2022 Information Security Management Systems



- ISO/IEC 27001:2022 specifies requirements for an Information Security Management System (ISMS)
- Annex A lists 93 reference controls, grouped in four themes:
5 Organizational (37), 6 People (8), 7 Physical (14), 8 Technological (34)
- ISO/IEC 27002:2022 gives implementation guidance for each control
- The organization decides which controls apply, based on risk assessment
– documented in the Statement of Applicability (SoA)
- Excluding a control requires a justification
“We don’t develop software” is often not a valid reason – scripts, automation and configuration count too

Source: ISO/IEC 27001:2022, ISO/IEC 27002:2022

Annex A – Secure Development Controls



- 8.25 Secure development life cycle
- 8.26 Application security requirements
- 8.27 Secure system architecture and engineering principles
- 8.28 Secure coding
- 8.29 Security testing in development and acceptance
- 8.30 Outsourced development
- Your ISMS project has to answer for each: does it apply, and how do we meet it?
- The following slides: what the control asks for (paraphrased), and questions to ask in your organization

Source: ISO/IEC 27001:2022 Annex A

Secure Development Life Cycle



Control 8.25:

Secure development life cycle

Rules for secure development of software and systems are established and applied. (paraphrased)

Do we have a documented secure development policy?

Which phases of our development process include security activities?

Who is responsible for security in each phase?

Are developers trained in secure development?

Source: ISO/IEC 27001:2022 Annex A 8.25

Application Security Requirements



Control 8.26:

Application security requirements

Information security requirements are identified, specified and approved when developing or acquiring applications. (paraphrased)

Where do our security requirements come from – risk assessment, legislation, GDPR, customers?

How are they written down – user stories, abuse cases, checklists?

Who approves them, and when in the project?

Do we use a reference, e.g. OWASP Top 10 or OWASP ASVS?

Source: ISO/IEC 27001:2022 Annex A 8.26, see also Session 1 OWASP

Secure System Architecture and Engineering Principles



Control 8.27:

Secure system architecture and engineering principles

Principles for engineering secure systems are established, documented, maintained and applied to all development. (paraphrased)

Which security principles do we follow – least privilege, fail-safe defaults, defense in depth?

Are they written down, or only in people's heads?

Do we do design reviews or threat modelling before implementation?

Would our products pass the “everything runs as root” test?

Source: ISO/IEC 27001:2022 Annex A 8.27, see also Session 2 principles and design



Control 8.28:

Secure coding

Secure coding principles are applied to software development. (paraphrased)

Do we have coding standards, and are they enforced?

Compiler warnings, linters and scanners – e.g. Ruff, Bandit, Semgrep?

Git hooks and CI – what is checked before code is merged?

Code review – does it include security, and who does it?

Source: ISO/IEC 27001:2022 Annex A 8.28, see also Session 4 coding standards and tools

Security Testing in Development and Acceptance



Control 8.29:

Security testing in development and acceptance

Security testing processes are defined and implemented in the development life cycle. (paraphrased)

Which security tests run automatically – static and dynamic analysis?

Do acceptance criteria include security requirements from 8.26?

Penetration testing before go-live – when, by whom?

How are findings tracked and fixed?

Source: ISO/IEC 27001:2022 Annex A 8.29, see also Session 3 code auditing and testing

Outsourced Development



Control 8.30:

Outsourced development

The organization directs, monitors and reviews activities related to outsourced system development.
(paraphrased)

Do contracts with suppliers include security requirements?

Do we have the right to audit, or to see test results?

How do we verify delivered code – same scanners and tests as our own?

Third-party libraries and dependencies – who is responsible for them?

Source: ISO/IEC 27001:2022 Annex A 8.30

Annex A Controls and This Course



- 8.25 Secure development life cycle – Session 5
- 8.26 Application security requirements – Session 1, OWASP Top 10
- 8.27 Secure system architecture and engineering principles – Session 2
- 8.28 Secure coding – Session 4
- 8.29 Security testing in development and acceptance – Session 3
- 8.30 Outsourced development – discussion in this session
- The controls describe **what** – the course has shown examples of **how**

Exercise: Where Are Our Gaps?



Imagine your manager asks about ISO/IEC 27001:2022 Annex A 8.25–8.30.
Can you help them, using the questions on the previous slides?

For each control:

- What do we already do – process, tools, documentation?
- What is missing or only informal?
- What would be the first concrete step?

Use what you shared in Session 4 about the tools you run today
Continues in the optional part at the end of the day

We cannot do a complete gap analysis project now, but hopefully your vocabulary has expanded.

I invite you to consider today's subjects and reflect on how you can change and mature your development work.

Questions?



Henrik Kramselund he/him han/ham hk@zencurity.com @kramse

You are always welcome to send me questions later via email

Mobile: +45 2026 6000

Email: hk@zencurity.com

Part 7: Optional: Exercises, questions



- I recommend doing exercises
- I use Linux and Kubernetes, but previous Linux and Unix knowledge is not needed
- It is recommended to use containers and virtual machines for the exercises
- Web security is a great example since the web server runs on top of operating systems and lower, plus it runs application frameworks, code by the developers etc.
- Security and most internet-related security work has the following requirements:
Network experience, Server experience, TCP/IP principles - often in more detail than a common user, Programming is an advantage, for automating things

Benchmarking tools



🔧 Kube-bench is the industry-standard tool to automate checking Kubernetes compliance with the Center for Internet Security (CIS) Benchmark.

Kube-bench makes it easy for operators to check whether each node in their Kubernetes cluster is configured according to security best practices.

Source: <https://info.aquasec.com/open-source>

- CIS Kubernetes V1.24 Benchmark v1.0.0 - 09-21-2022 – other versions exist
- CIS Docker Benchmark v1.5.0 - 12-28-2022

Tool example kube-bench



```
hlk@timon:~/bin/kube-bench/kube-bench$ kubectl logs kube-bench-gdf62
[PASS] 1.1.7 Ensure that the etcd pod specification file permissions are set to 600 or more restrictive (Automated)
[PASS] 1.1.8 Ensure that the etcd pod specification file ownership is set to root:root (Automated)
[WARN] 1.1.9 Ensure that the Container Network Interface file permissions are set to 600 or more restrictive (Manual)
[WARN] 1.1.10 Ensure that the Container Network Interface file ownership is set to root:root (Manual)
[PASS] 1.1.11 Ensure that the etcd data directory permissions are set to 700 or more restrictive (Automated)
[FAIL] 1.1.12 Ensure that the etcd data directory ownership is set to etcd:etcd (Automated)
...
== Summary policies ==
0 checks PASS
0 checks FAIL
35 checks WARN
0 checks INFO

== Summary total ==
63 checks PASS
10 checks FAIL
58 checks WARN
0 checks INFO
```

- <https://github.com/aquasecurity/kube-bench> also check out Lynis <https://cisofy.com/lynis/>